

Содержание

1. Цель работы	3
2. Теоретическое введение	3
2.1. Многослойный перцептрон	3
2.1.1. Соглашение о числе слоёв	3
2.2. Вычислительный граф MLP	4
2.3. Функции активации	4
2.3.1. Сигмоида	4
2.3.2. Гиперболический тангенс	4
2.3.3. ReLU	4
2.4. Функции ошибки	5
2.4.1. Для регрессии — MSE	5
2.4.2. Для классификации — кросс-энтропия	5
2.5. Оптимизация параметров нейронной сети	5
2.5.1. Градиентный спуск	5
2.5.2. Улучшения: Momentum и RMSProp	5
2.5.3. Adam (Adaptive Moment Estimation)	6
2.5.3.1. Формулы Adam	6
2.5.3.2. Значения по умолчанию в PyTorch	7
2.6. Ранняя остановка	7
2.7. Стандартизация	7
2.8. Кросс-валидация	7
3. Программная реализация	8
3.1. Построение MLP	8
3.2. Обучение модели	8
3.3. Генерация данных	9
3.3.1. Регрессия	9
3.3.2. Классификация	9
3.4. Метрики	10
3.4.1. MSE (регрессия)	10
3.4.2. Accuracy (классификация)	10
3.4.3. Confusion matrix	10
4. Часть I. Регрессия на выборках из задания 1	10
4.1. Описание эксперимента	10
4.2. Сводные таблицы Test MSE	10
4.2.1. Полином + равномерный шум	11
4.2.2. $x \sin(2\pi x)$ + нормальный шум	11
4.3. Визуализация лучших моделей	11

4.3.1. Полином	12
4.3.2. $x \sin(2\pi x)$	13
4.4. Демонстрация переобучения	13
5. Часть II. Классификация на выборках из задания 2	14
5.1. Описание эксперимента	14
5.2. Визуализация датасетов	15
5.3. Сводные таблицы Test Accuracy	16
5.4. Визуализация лучших моделей	16
5.4.1. Circles	16
5.4.2. XOR	17
5.4.3. Blobs	18
5.4.4. Spiral	19
6. Оценка минимального размера обучающей выборки	19
6.1. Постановка задачи	19
6.2. Результаты	20
7. K-fold кросс-валидация	20
7.1. Постановка задачи	20
7.2. Результаты	20
8. Выводы	20
9. Структура проекта	21
10. Репозиторий	21

1. Цель работы

Цель лабораторной работы — реализация многослойного перцептрона (МСП) средствами библиотеки `torch.nn` и исследование его возможностей при решении задач регрессии и классификации.

В работе исследуются:

- три функции активации (Sigmoid, Tanh, ReLU);
- архитектуры с числом линейных слоёв от 2 до 4;
- число нейронов в скрытом слое от 1 до 5;
- регрессия на выборках из задания 1 (полином 3-й степени и осциллирующая функция $x \sin(2\pi x)$);
- классификация на выборках из задания 2 (Circles, XOR, Blobs, Spiral);
- оценка минимального размера обучающей выборки для достижения ассурасы $\geq 90\%$;
- K-fold кросс-валидация для подбора функции активации и числа слоёв.

2. Теоретическое введение

2.1. Многослойный перцептрон

Многослойный перцептрон (MLP — Multilayer Perceptron) — нейронная сеть прямого распространения, состоящая из входного, одного или нескольких скрытых и выходного слоёв.

Каждый слой l выполняет преобразование:

$$h^{(l)} = \varphi(W^{(l)}h^{(l-1)} + b^{(l)}),$$

где $W^{(l)}$ — матрица весов, $b^{(l)}$ — вектор смещений, φ — функция активации, $h^{(0)} = x$ — вход сети.

Без нелинейности φ композиция линейных слоёв сводится к одному линейному преобразованию, поэтому функция активации критически необходима для моделирования сложных зависимостей.

2.1.1. Соглашение о числе слоёв

Под числом слоёв понимается число **линейных** слоёв вместе с выходным. Таким образом:

- 2 слоя = 1 скрытый слой;
- 3 слоя = 2 скрытых слоя;
- 4 слоя = 3 скрытых слоя.

2.2. Вычислительный граф MLP

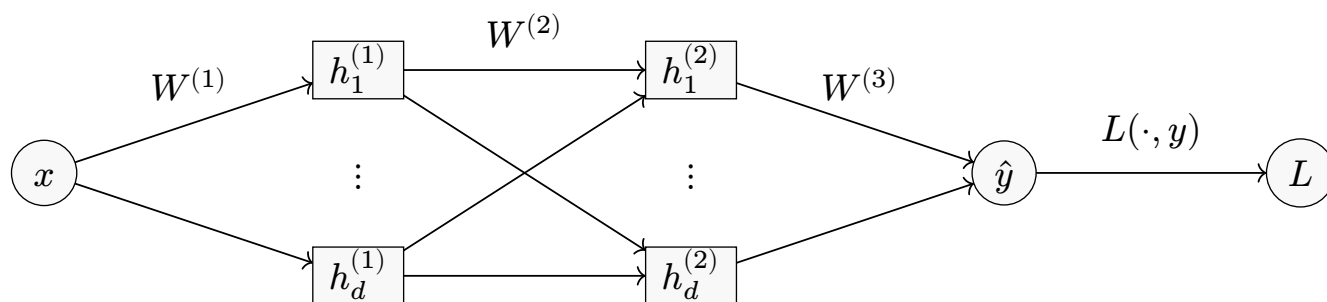


Рис. 1. Вычислительный граф MLP с двумя скрытыми слоями. Каждый скрытый узел включает линейное преобразование и функцию активации φ .

2.3. Функции активации

Нелинейная функция активации вводится после каждого линейного преобразования скрытого слоя. В работе рассматриваются три варианта.

2.3.1. Сигмоида

$$\sigma(a) = \frac{1}{1 + e^{-a}}, \quad \sigma(a) \in (0, 1).$$

Гладкая, монотонная, с насыщением при $|a| \rightarrow \infty$. Может приводить к затуханию градиента при большой глубине сети.

2.3.2. Гиперболический тангенс

$$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}, \quad \tanh(a) \in (-1, 1).$$

Центрирован относительно нуля, что обычно ускоряет сходимость по сравнению с сигмойдой, но также подвержен затуханию градиента.

2.3.3. ReLU

$$\text{ReLU}(a) = \max(0, a).$$

Вычислительно эффективна, не имеет насыщения при $a > 0$. Градиент равен нулю при $a < 0$ (проблема «мёртвых нейронов»).

Sigmoid

Выход: $(0, 1)$

Гладкая, насыщенная

Затухание градиента

Tanh

Выход: $(-1, 1)$

Центрирована в нуле

Затухание градиента

ReLU

Выход: $[0, +\infty)$

Быстрая, без насыщения

«Мёртвые нейроны»

2.4. Функции ошибки

2.4.1. Для регрессии — MSE

$$E(w) = \frac{1}{N} \sum_{i=1}^N \left(\tilde{f}(w, x^{(i)}) - y^{(i)} \right)^2,$$

где $\tilde{f}(w, x)$ — выход сети с параметрами w .

2.4.2. Для классификации — кросс-энтропия

При использовании двух выходных нейронов и `softmax`:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=0}^1 y_c^{(i)} \log \hat{y}_c^{(i)},$$

где $\hat{y}_c = \text{softmax}(z)_c = \frac{e^{z_c}}{e^{z_0} + e^{z_1}}$ — предсказанная вероятность класса c .

2.5. Оптимизация параметров нейронной сети

При обучении нейронной сети мы решаем задачу минимизации функции потерь, и для этого используем методы оптимизации.

2.5.1. Градиентный спуск

Простейший метод — **градиентный спуск** (Gradient Descent). На каждом шаге t параметры обновляются в направлении антиградиента:

$$w_{t+1} = w_t - \eta \cdot \nabla L(w_t),$$

где η — learning rate (шаг обучения), ∇L — градиент функции потерь по параметрам.

Однако у обычного градиентного спуска есть существенные проблемы:

1. **Один learning rate для всех параметров** — некоторые веса нуждаются в большем шаге, а другие — в меньшем.
2. **Медленная сходимост**ь — в «оврагах» ландшафта потерь метод колеблется поперёк оврага, но медленно продвигается вдоль.
3. **Чувствительность к масштабу** — если признаки сильно различаются по масштабу, градиенты по разным компонентам отличаются на порядки.

2.5.2. Улучшения: Momentum и RMSProp

Для преодоления этих проблем были предложены два ключевых усовершенствования.

Momentum — учитывает инерцию: к текущему градиенту добавляется экспоненциально затухающее среднее предыдущих градиентов. Это сглаживает колебания и ускоряет движение в устойчивом направлении.

RMSProp — адаптирует шаг для каждого параметра отдельно: делит learning rate на скользящее среднее квадратов градиентов. Параметры с большими градиентами получают меньший шаг, а с маленькими — больший.

2.5.3. Adam (Adaptive Moment Estimation)

Алгоритм **Adam** объединяет идеи Momentum и RMSProp. Он:

- накапливает экспоненциально взвешенное среднее градиентов (первый момент);
- накапливает экспоненциально взвешенное среднее квадратов градиентов (второй момент).

Т.е.:

- если градиент стабильно направлен в одну сторону — Adam ускоряется;
- если градиент колеблется (шум) — Adam сглаживает обновления;
- если компонента градиента часто велика — шаг по ней уменьшается;
- если компонента градиента мала — шаг увеличивается.

2.5.3.1. Формулы Adam

Обозначим $g_t = \nabla L(w_t)$ — градиент на шаге t .

Шаг 1. Обновление экспоненциального среднего градиента (первый момент):

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t.$$

Здесь m_t — оценка **среднего направления** градиента. β_1 (обычно 0.9) определяет, как сильно учитываются прошлые градиенты: чем ближе к 1, тем сильнее сглаживание.

Шаг 2. Обновление экспоненциального среднего квадрата градиента (второй момент):

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2.$$

Здесь v_t — оценка **дисперсии** (масштаба) градиента. β_2 (обычно 0.999) задаёт длину «памяти» для квадратов градиентов.

Шаг 3. Коррекция смещения (bias correction). На ранних шагах m_t и v_t смещены к нулю, поскольку инициализируются нулями. Коррекция:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

При малых t знаменатель $(1 - \beta^t)$ мал, что увеличивает оценку и компенсирует начальное смещение. С ростом t коррекция исчезает.

Шаг 4. Обновление параметров:

$$w_{t+1} = w_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

Здесь:

- η — базовый learning rate (в нашей работе 0.02 для регрессии, 0.03 для классификации);
- $\varepsilon \approx 10^{-8}$ — малая константа для защиты от деления на ноль;
- дробь $\hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$ — это **адаптивный шаг**: направление задаётся первым моментом, а масштаб регулируется вторым.

2.5.3.2. Значения по умолчанию в PyTorch

В `torch.optim.Adam` используются: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$. На практике эти значения почти никогда не меняют — единственный важный гиперпараметр для настройки — это η .

2.6. Ранняя остановка

В процессе обучения на каждой эпохе вычисляются потери на обучающей и тестовой частях. Сохраняется состояние модели с наименьшей тестовой ошибкой. Это позволяет предотвратить переобучение: модель запоминает лучшую обобщающую способность.

2.7. Стандартизация

Перед обучением признаки и (для регрессии) целевые значения стандартизируются:

$$\hat{x} = \frac{x - \mu}{\sigma},$$

где μ и σ вычисляются по обучающей части. Стандартизация ускоряет сходимость и улучшает устойчивость обучения.

2.8. Кросс-валидация

Выборка S случайно делится на K равных подмножеств $S_1, \dots, S_K$. На каждой итерации i :

- обучающая выборка: $S'_i = \bigcup_{j \neq i} S_j$;
- тестовая (валидационная) выборка: $T'_i = S_i$.

Кросс-валидация:

1. K раз проводим обучение и тестирование (валидацию) на S'_i, T'_i ($i = 1, \dots, K$);
2. усредняем результаты проверок;
3. выбираем модель с наименьшей ошибкой.

3. Программная реализация

3.1. Построение MLP

В PyTorch нейросеть строится как последовательность модулей `nn.Sequential`. Каждый скрытый слой состоит из линейного преобразования `nn.Linear` и функции активации.

Исходный файл: mlp.py

```
ACTIVATIONS = {
    'sigmoid': nn.Sigmoid,
    'tanh':    nn.Tanh,
    'relu':    nn.ReLU,
}

def build_mlp(input_dim, output_dim, hidden_layers, hidden_dim, activation):
    layers = []
    in_f = input_dim
    act_cls = ACTIVATIONS[activation]

    for _ in range(hidden_layers):
        layers.append(nn.Linear(in_f, hidden_dim))
        layers.append(act_cls())
        in_f = hidden_dim

    layers.append(nn.Linear(in_f, output_dim))
    return nn.Sequential(*layers).to(device)
```

Параметры:

- `input_dim` — размерность входа (1 для регрессии, 2 для классификации);
- `output_dim` — размерность выхода (1 для регрессии, 2 для классификации);
- `hidden_layers` — количество скрытых слоёв (1, 2 или 3);
- `hidden_dim` — число нейронов в каждом скрытом слое (от 1 до 5);
- `activation` — функция активации (`sigmoid`, `tanh`, `relu`).

3.2. Обучение модели

Функция `train_model` реализует универсальное обучение: для регрессии используется `MSELoss`, для классификации — `CrossEntropyLoss`. На каждой эпохе сохраняется лучшее состояние модели по `val loss`.

Исходный файл: mlp.py

```

def train_model(model, X_train, y_train, X_val, y_val,
                 task='regression', epochs=300, lr=0.01):
    loss_fn = nn.MSELoss() if task == 'regression' else nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    history = {'train_loss': [], 'val_loss': []}
    best_state = None
    best_val = float('inf')

    for _ in range(epochs):
        model.train()
        optimizer.zero_grad()
        out = model(X_train)
        loss = loss_fn(out, y_train)
        loss.backward()
        optimizer.step()

        model.eval()
        with torch.no_grad():
            val_out = model(X_val)
            val_loss = loss_fn(val_out, y_val)

        history['train_loss'].append(float(loss.item()))
        history['val_loss'].append(float(val_loss.item()))

        if val_loss.item() < best_val:
            best_val = float(val_loss.item())
            best_state = copy.deepcopy(model.state_dict())

    if best_state is not None:
        model.load_state_dict(best_state)
    return history

```

3.3. Генерация данных

3.3.1. Регрессия

Используются две функции из задания 1:

1. Случайный полином 3-й степени: $f(x) = ax^3 + bx^2 + cx + d$, сгенерированные коэффициенты: $a \approx 0.751$, $b \approx 2.383$, $c \approx 1.654$, $d \approx -1.649$.
2. Осциллирующая функция: $f(x) = x \sin(2\pi x)$.

Для полинома добавляется равномерный шум, для тригонометрической функции — нормальный.

3.3.2. Классификация

Используются четыре типа выборок из задания 2: Circles, XOR, Blobs, Spiral.

3.4. Метрики

3.4.1. MSE (регрессия)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2.$$

3.4.2. Accuracy (классификация)

$$\text{accuracy} = \frac{T_P + T_N}{T_P + F_P + T_N + F_N}.$$

3.4.3. Confusion matrix

$$\begin{pmatrix} T_N & F_P \\ F_N & T_P \end{pmatrix}$$

4. Часть I. Регрессия на выборках из задания 1

4.1. Описание эксперимента

Для каждой из двух функций (полином и $x \sin(2\pi x)$) строится MLP с перебором всех комбинаций:

- функция активации: `sigmoid`, `tanh`, `relu`;
- число скрытых слоёв: 1, 2, 3 (линейных слоёв: 2, 3, 4);
- число нейронов в скрытом слое: 1, 2, 3, 4, 5.

Параметры обучения:

- $N = 280$ объектов, разбиение 75/25;
- `epochs = 350`, `learning rate = 0.02`;
- оптимизатор `Adam`, ранняя остановка по `val loss`.

Всего $3 \times 3 \times 5 = 45$ архитектур на каждую функцию.

4.2. Сводные таблицы Test MSE

В таблицах строки — число линейных слоёв (2, 3, 4), столбцы — число нейронов в скрытом слое ($N = 1, \dots, 5$). Значения — Test MSE.

4.2.1. Полином + равномерный шум

Активация	Слоёв	N=1	N=2	N=3	N=4	N=5
Sigmoid	2	0.0681	0.0271	0.0207	0.0213	0.0183
Sigmoid	3	0.0523	0.0251	0.0230	0.0230	0.0210
Sigmoid	4	0.0354	0.0289	0.0300	0.0233	0.0242
Tanh	2	0.0230	0.0275	0.0174	0.0203	0.0165
Tanh	3	0.0253	0.0202	0.0158	0.0132	0.0178
Tanh	4	0.0254	0.0192	0.0129	0.0174	0.0122
ReLU	2	0.0490	0.5333	0.0372	0.0150	0.0395
ReLU	3	1.3477	1.3444	1.3831	0.0147	0.0173
ReLU	4	1.3445	0.0371	1.3445	0.0163	0.0160

Таблица 1. Test MSE для полинома 3-й степени с равномерным шумом.

4.2.2. $x \sin(2\pi x)$ + нормальный шум

Активация	Слоёв	N=1	N=2	N=3	N=4	N=5
Sigmoid	2	0.1289	0.0392	0.0319	0.0384	0.0385
Sigmoid	3	0.1045	0.0342	0.0259	0.0229	0.0229
Sigmoid	4	0.1004	0.1006	0.0199	0.0221	0.0186
Tanh	2	0.1091	0.0321	0.0299	0.0190	0.0209
Tanh	3	0.1014	0.0202	0.0175	0.0175	0.0093
Tanh	4	0.1000	0.0205	0.0118	0.0093	0.0035
ReLU	2	0.1287	0.0868	0.0471	0.0433	0.0430
ReLU	3	0.1287	0.0867	0.1093	0.0862	0.0323
ReLU	4	0.1287	0.1287	0.1287	0.0083	0.0031

Таблица 2. Test MSE для $x \sin(2\pi x)$ с нормальным шумом.

4.3. Визуализация лучших моделей

Для лучшей модели по каждой функции активации строятся:

- график train/test loss по эпохам;
- график истинной функции и предсказания сети.

Лучшие модели для полинома:

- Sigmoid: 2 слоя, 5 нейронов, MSE = 0.0183;
- Tanh: 4 слоя, 5 нейронов, MSE = 0.0122;
- ReLU: 3 слоя, 4 нейрона, MSE = 0.0147.

Лучшие модели для $x \sin(2\pi x)$:

- Sigmoid: 4 слоя, 5 нейронов, MSE = 0.0186;
- Tanh: 4 слоя, 5 нейронов, MSE = 0.0035;
- ReLU: 4 слоя, 5 нейронов, MSE = 0.0031.

4.3.1. Полином

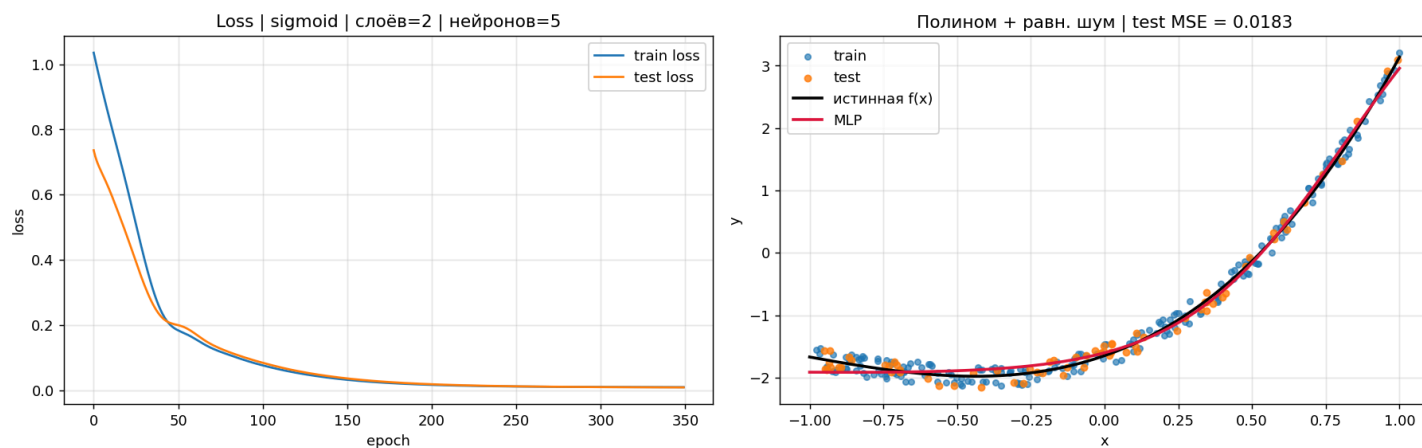


Рис. 2. Лучшая модель (Sigmoid) для полинома.

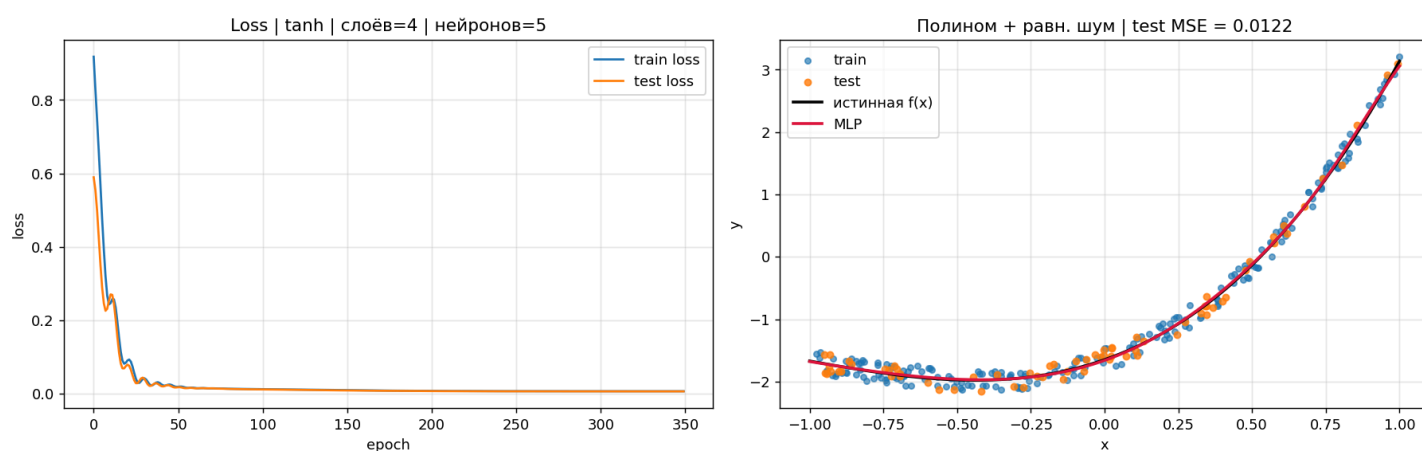


Рис. 3. Лучшая модель (Tanh) для полинома.

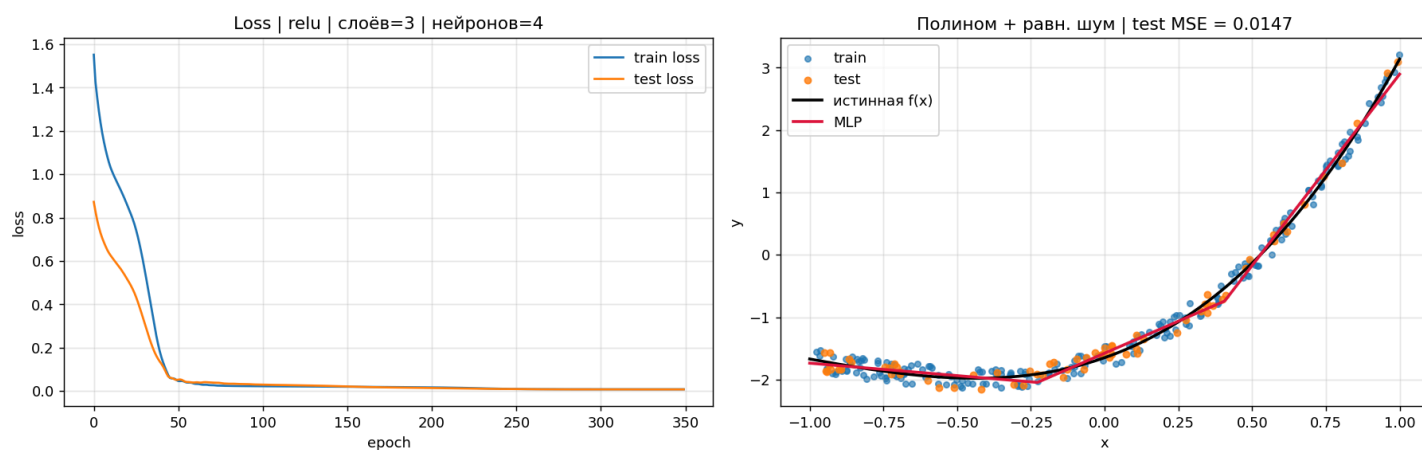


Рис. 4. Лучшая модель (ReLU) для полинома.

4.3.2. $x \sin(2\pi x)$

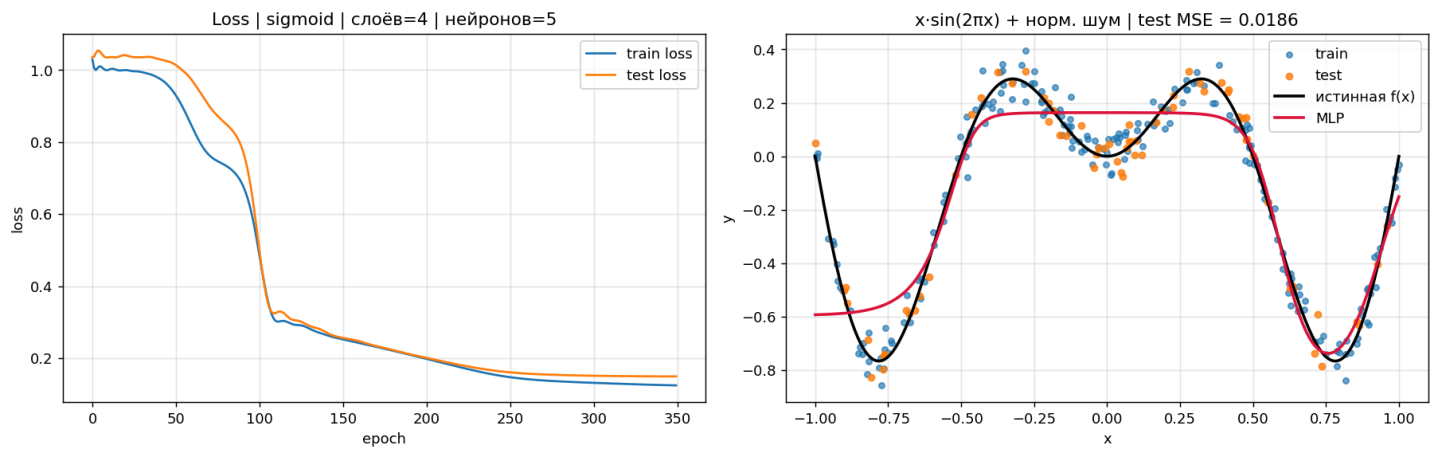


Рис. 5. Лучшая модель (Sigmoid) для $x \sin(2\pi x)$.

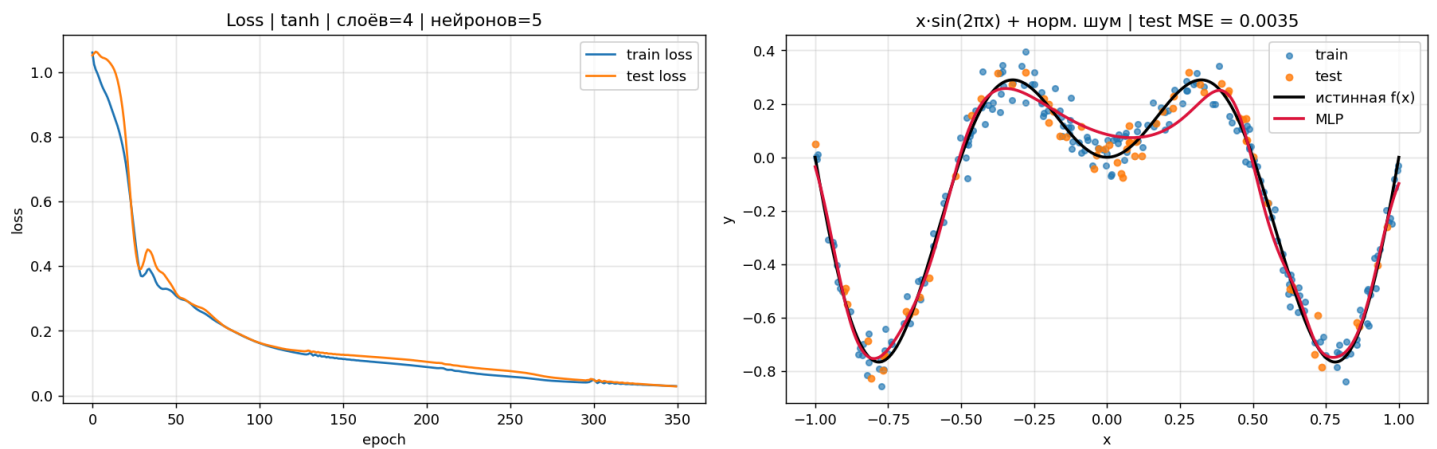


Рис. 6. Лучшая модель (Tanh) для $x \sin(2\pi x)$.

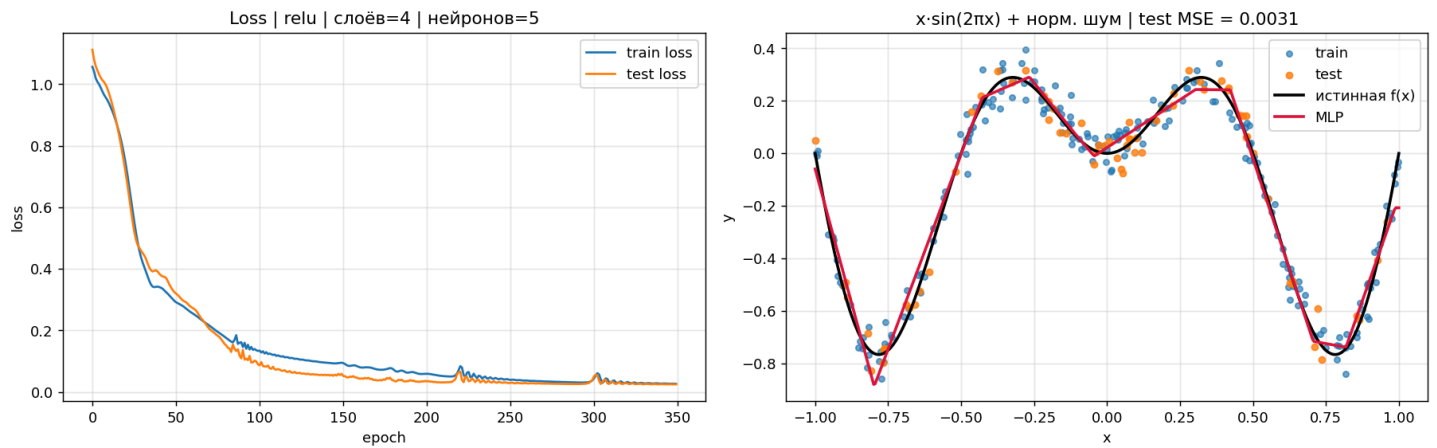


Рис. 7. Лучшая модель (ReLU) для $x \sin(2\pi x)$.

4.4. Демонстрация переобучения

Для наглядной демонстрации переобучения берётся:

- осциллирующая функция $f(x) = x \sin(2\pi x)$;
- маленькая обучающая подвыборка ($n_{\text{train}} = 10$);
- избыточная сеть (8 скрытых слоёв, 5 нейронов, $\tanh$);
- обучение без ранней остановки (5000 эпох).

Результаты:

- Train MSE = 0.000052;
- Test MSE = 0.046685;
- лучшая эпоха по val loss: 1738.

Train loss продолжает падать, а test loss после лучшей эпохи начинает расти — классический признак переобучения.

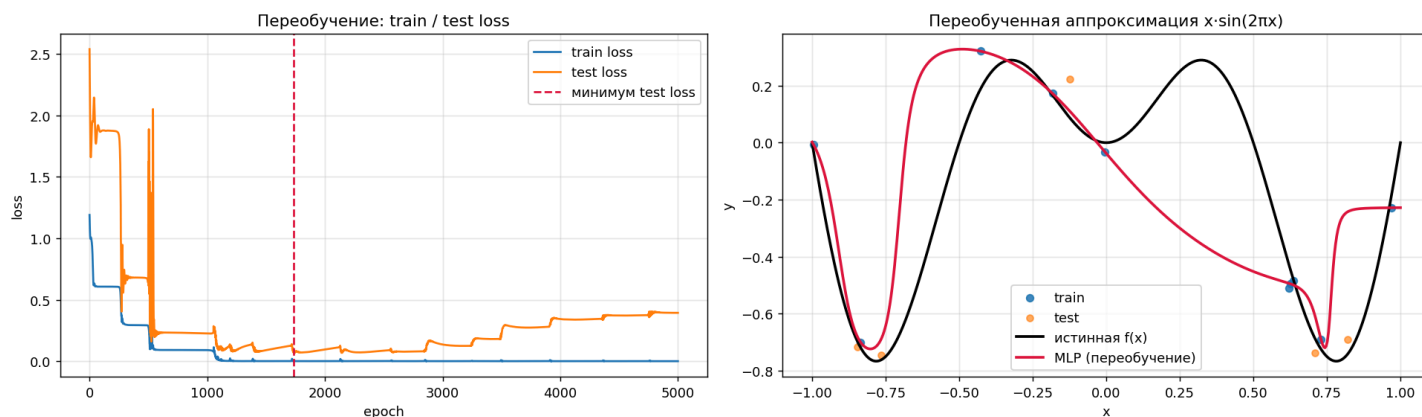


Рис. 8. Демонстрация переобучения: train loss падает, test loss растёт.

5. Часть II. Классификация на выборках из задания 2

5.1. Описание эксперимента

Для каждого из четырёх датасетов (Circles, XOR, Blobs, Spiral) выполняется перебор архитектур аналогично регрессии:

- функция активации: `sigmoid`, `tanh`, `relu`;
- число скрытых слоёв: 1, 2, 3;
- число нейронов в скрытом слое: 1, 2, 3, 4, 5.

Параметры:

- $N = 500$ объектов, разбиение 75/25;
- `epochs = 500`, `learning rate = 0.03`;
- выходной слой — 2 нейрона, `CrossEntropyLoss`.

5.2. Визуализация датасетов

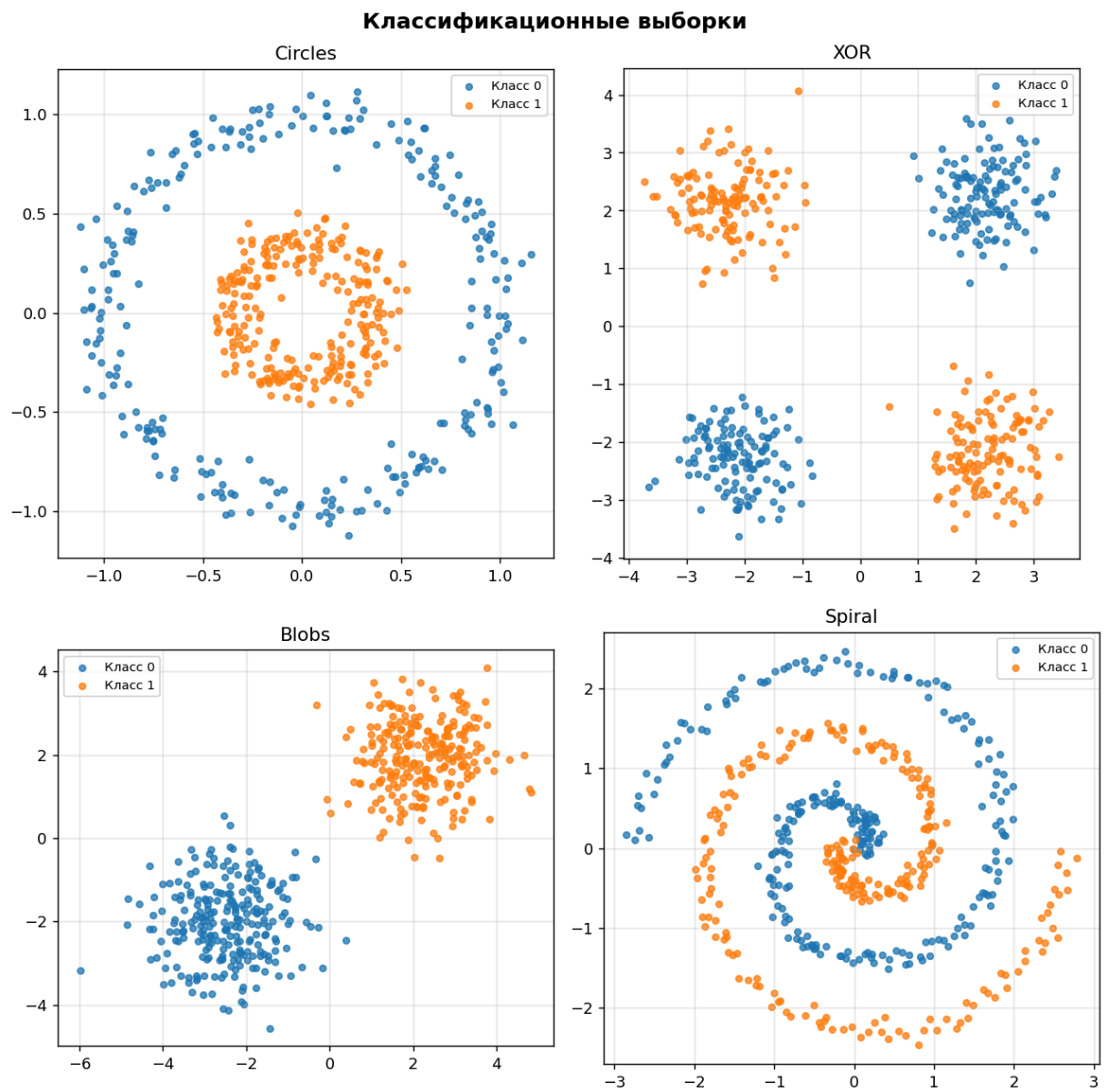


Рис. 9. Четыре классификационных датасета.

5.3. Сводные таблицы Test Accuracy

Датасет	Активация	N=1	N=2	N=3	N=4	N=5
Circles	Sigmoid	0.6000	0.8800	1.0000	1.0000	1.0000
Circles	Tanh	0.6080	0.8320	1.0000	1.0000	1.0000
Circles	ReLU	0.6240	0.5520	0.9920	0.5520	1.0000
XOR	Sigmoid	0.7680	1.0000	1.0000	1.0000	0.9920
XOR	Tanh	0.7280	1.0000	1.0000	0.9920	0.9920
XOR	ReLU	0.5200	0.7360	1.0000	1.0000	1.0000
Blobs	Sigmoid	1.0000	1.0000	1.0000	1.0000	1.0000
Blobs	Tanh	1.0000	1.0000	1.0000	1.0000	1.0000
Blobs	ReLU	0.5280	0.5280	1.0000	1.0000	1.0000
Spiral	Sigmoid	0.6240	0.6240	0.5920	0.6880	0.9440
Spiral	Tanh	0.6320	0.6240	0.6240	0.8720	0.8800
Spiral	ReLU	0.5280	0.5680	0.6480	0.6480	0.7520

Таблица 3. Test accuracy для классификации (при 4 линейных слоях).

5.4. Визуализация лучших моделей

Для лучшей модели на каждом датасете строятся: график loss, граница решений и confusion matrix.

5.4.1. Circles

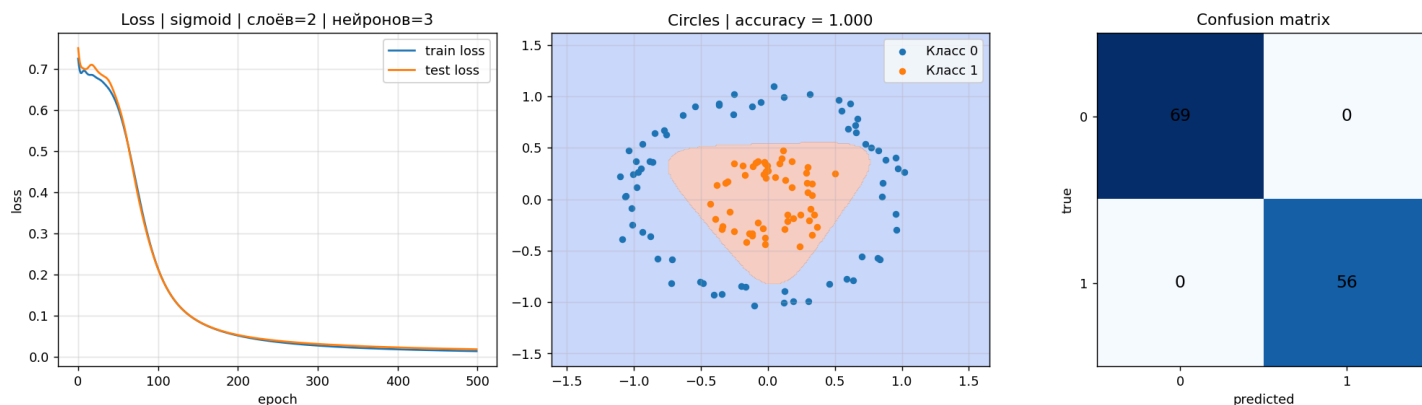


Рис. 10. Лучшая модель (Sigmoid) для Circles.

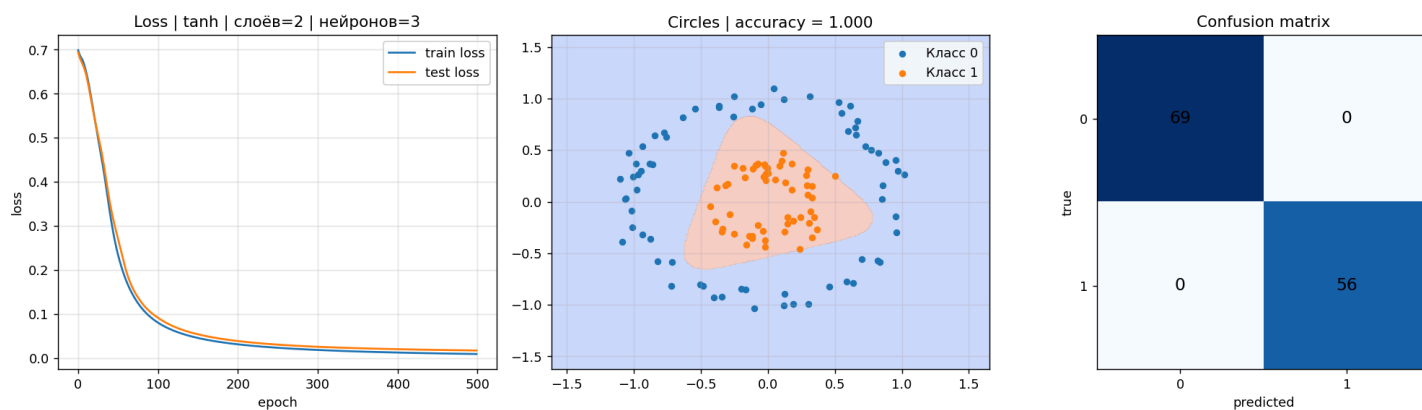


Рис. 11. Лучшая модель (Tanh) для Circles.

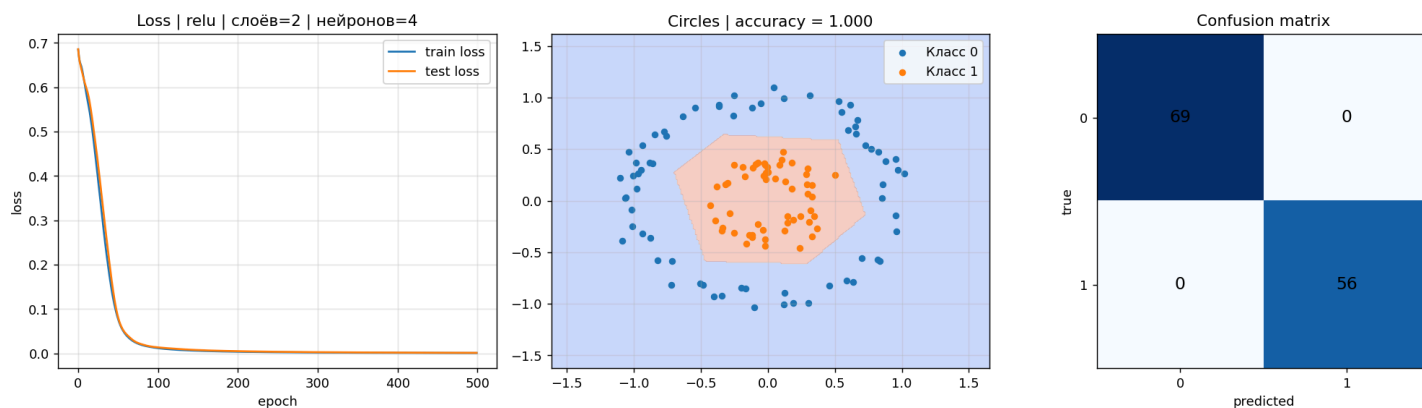


Рис. 12. Лучшая модель (ReLU) для Circles.

5.4.2. XOR

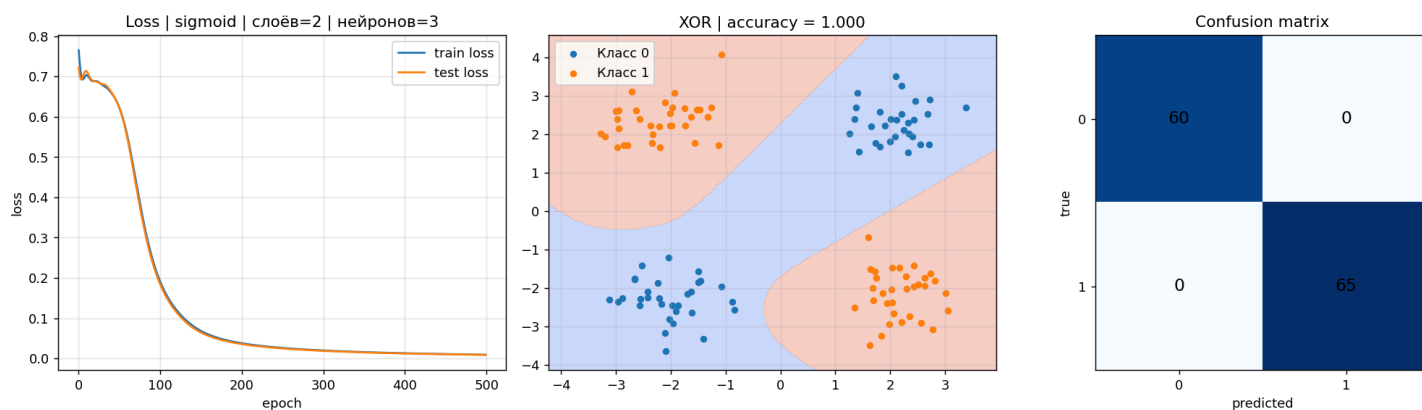


Рис. 13. Лучшая модель (Sigmoid) для XOR.

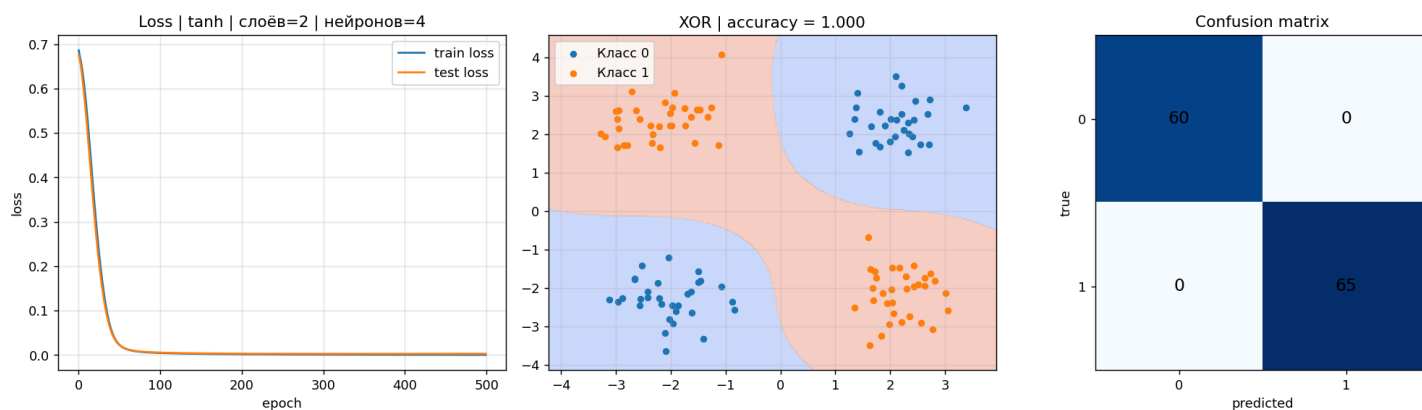


Рис. 14. Лучшая модель (Tanh) для XOR.

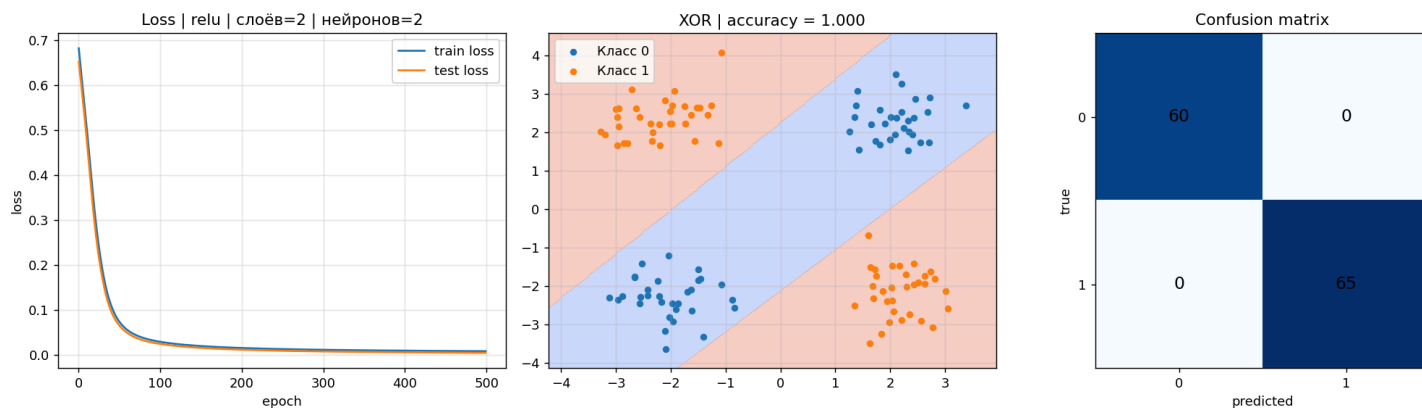


Рис. 15. Лучшая модель (ReLU) для XOR.

5.4.3. Blobs

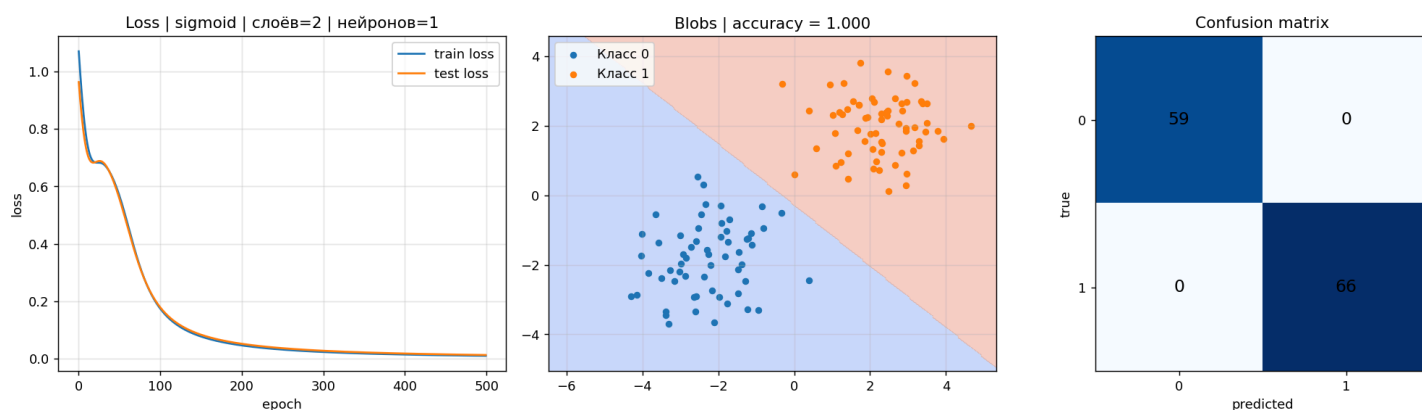


Рис. 16. Лучшая модель (Sigmoid) для Blobs.

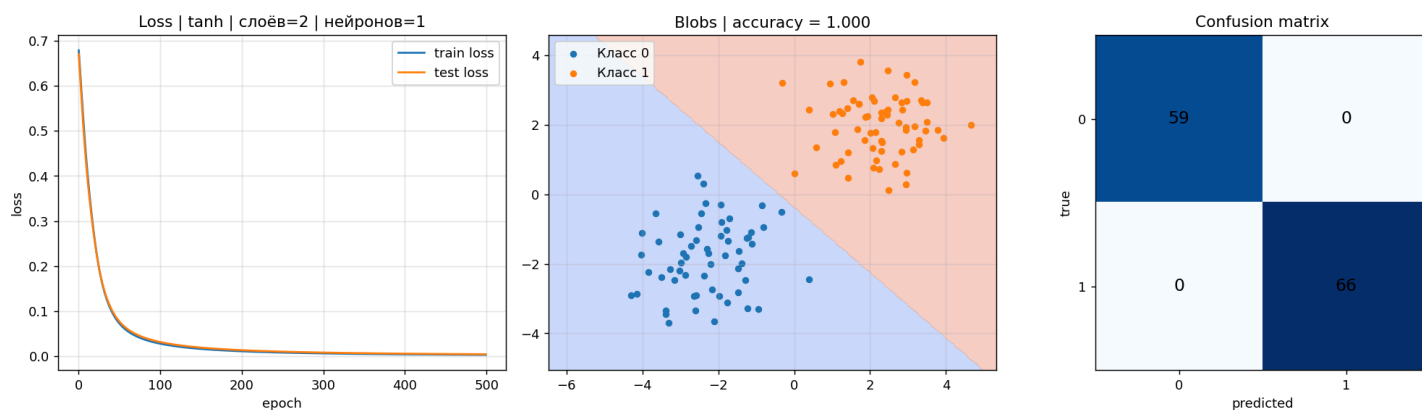


Рис. 17. Лучшая модель (Tanh) для Blobs.

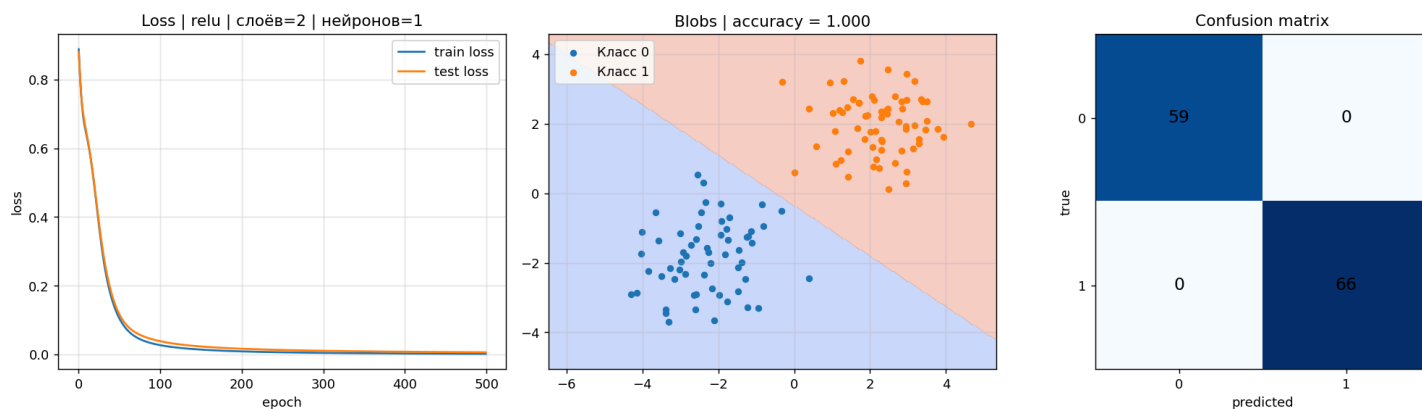


Рис. 18. Лучшая модель (ReLU) для Blobs.

5.4.4. Spiral

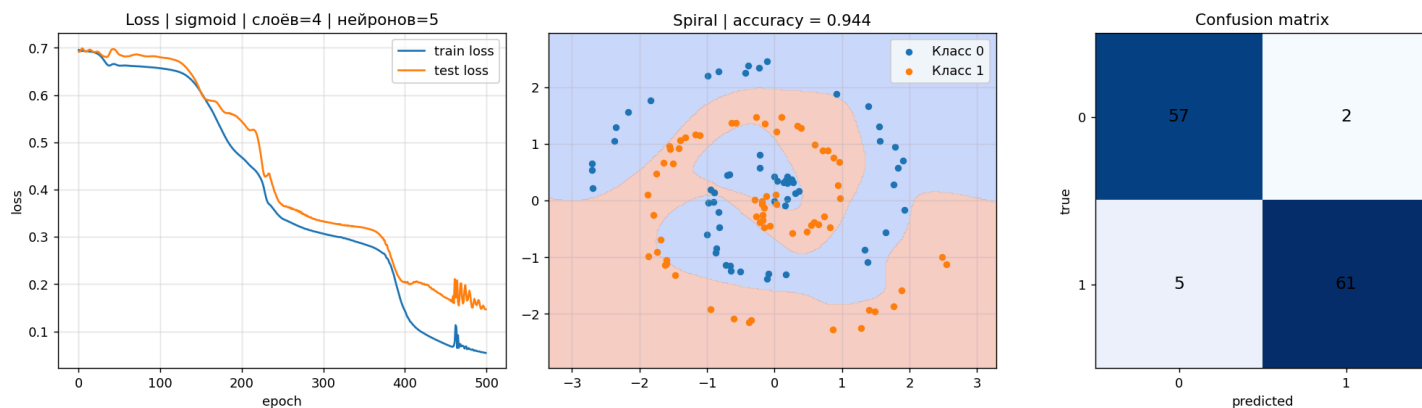


Рис. 19. Лучшая модель (Sigmoid) для Spiral.

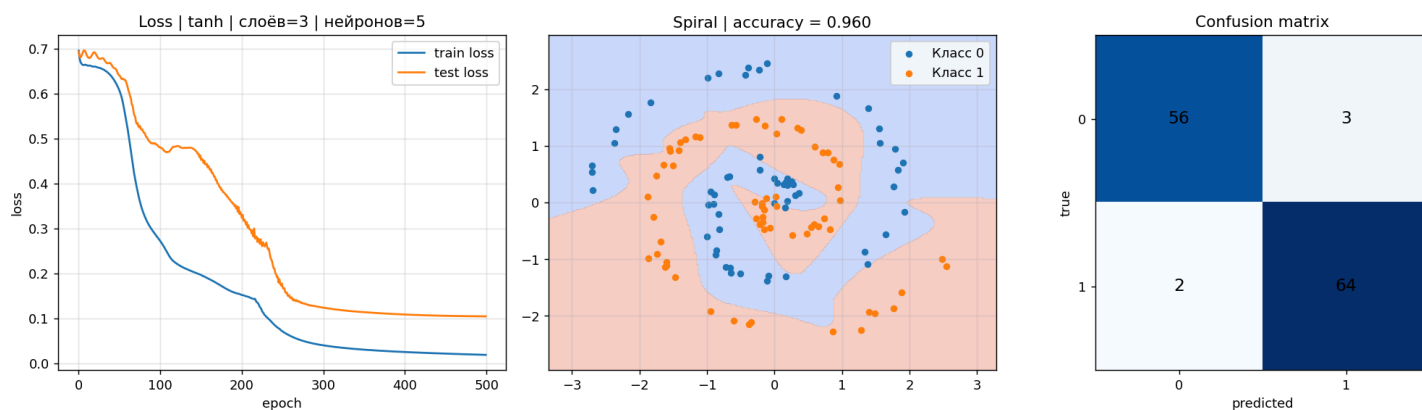


Рис. 20. Лучшая модель (Tanh) для Spiral.

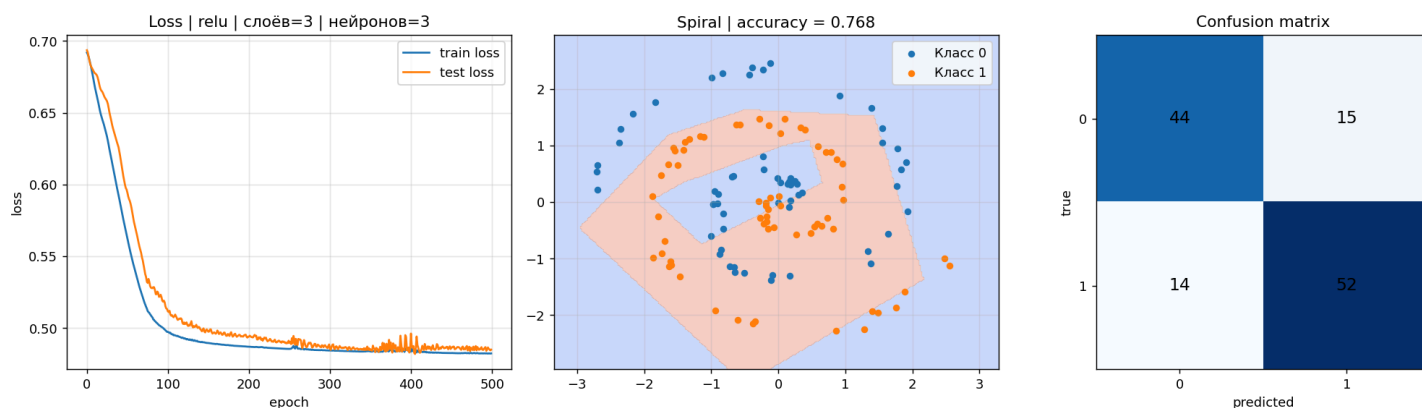


Рис. 21. Лучшая модель (ReLU) для Spiral.

6. Оценка минимального размера обучающей выборки

6.1. Постановка задачи

Найти минимальное количество элементов в обучающей выборке, при котором классификатор достигает accuracy $\geq 90\%$ на тестовой выборке.

Фиксированные параметры:

- функция активации: `tanh`;
- число линейных слоёв: 4;
- число нейронов: 5;
- `epochs = 250`, `lr = 0.03`.

Доля обучающей части варьируется от 10% до 95% с шагом 5%.

6.2. Результаты

Датасет	Активация	Слоёв	Нейронов	Min frac	Min N
Circles	tanh	4	5	0.10	50
XOR	tanh	4	5	0.10	50
Blobs	tanh	4	5	0.10	50
Spiral	tanh	4	5	0.40	200

Таблица 4. Минимальный размер обучающей выборки для accuracy $\geq 90\%$.

7. K-fold кросс-валидация

7.1. Постановка задачи

Для каждого датасета из задания 2 определить наилучшие значения:

1. функции активации (sigmoid / tanh / ReLU);
2. числа линейных слоёв (2, 3 или 4).

Число нейронов фиксировано: $d = 5$. Используется 5-fold кросс-валидация.

7.2. Результаты

Датасет	Лучшая активация	Слоёв	Mean acc	Std acc
Circles	sigmoid	2	1.0000	0.0000
XOR	tanh	2	1.0000	0.0000
Blobs	sigmoid	2	1.0000	0.0000
Spiral	tanh	4	0.9820	0.0205

Таблица 5. Лучшие результаты 5-fold кросс-валидации.

8. Выводы

По результатам экспериментов:

1. Многослойный перцептрон, реализованный средствами `torch.nn`, успешно решает как задачи регрессии, так и бинарной классификации.

2. Для регрессии MLP аппроксимирует полином 3-й степени и осциллирующую функцию $x \sin(2\pi x)$. Качество аппроксимации зависит от числа нейронов и глубины сети.
3. Функция активации влияет на скорость обучения и качество:
 - `tanh` и `relu` обычно дают лучшие результаты;
 - `sigmoid` медленнее сходится из-за затухания градиента.
4. Для простых задач классификации (Blobs) высокая точность достигается даже при минимальной архитектуре (2 слоя, 1 нейрон).
5. Для сложных задач (Spiral) критически важны глубина сети и число нейронов. Функция активации ReLU и Tanh показывают лучшие результаты.
6. Переобучение наглядно проявляется при малом размере обучающей выборки ($n = 10$) и избыточной сети: `train loss` падает, а `test loss` начинает расти.
7. Кросс-валидация позволяет надёжно выбрать оптимальную архитектуру модели, усредняя результаты по нескольким разбиениям.
8. Минимальный размер обучающей выборки для accuracy $\geq 90\%$ зависит от сложности геометрии данных: для Blobs достаточно малого числа объектов, для Spiral требуется значительно больше.

9. Структура проекта

```
SMGMO/  
├─ lab3/  
│   ├── main.typ          # отчёт (Typst)  
│   ├── config.py         # параметры экспериментов (через env-переменные)  
│   ├── generators.py     # генерация данных (регрессия + классификация)  
│   ├── mlp.py            # MLP: построение, обучение, предсказание, метрики  
│   ├── block1_main.py    # Блок 1: регрессия + переобучение  
│   ├── block2_main.py    # Блок 2: классификация + min N + кросс-валидация  
│   └─ gui_launcher.py    # GUI-лаунчер (Tkinter)
```

10. Репозиторий

Исходный код проекта доступен на GitHub: <https://github.com/Q1zin/SMGMO.git>